

C# Assignment

In this assignment, you will create a simple vehicle management system using C#. You will define a base class for vehicles and derived classes for specific types of vehicles, implementing inheritance and method overriding.

Task:

1. Define Vehicle Base Class:

Create a base class named Vehicle to represent general vehicles.

Add properties to the class: Make (string type), Model (string type), and Year (integer type).

Implement a constructor in the Vehicle class to initialize these properties.

Add a virtual method named GetInfo() to display the vehicle's details.

2. Define Derived Classes:

Create a derived class named Car that inherits from Vehicle.

Create a derived class named Motorcycle that inherits from Vehicle.

Override the `GetInfo()` method in both derived classes to include specific details (e.g., type of vehicle).

3. Input Handling:

Use the `Console.ReadLine()` method to prompt the user to enter the make, model, and year of the vehicle.

4. Create Instances and Display Information:

Instantiate the `Car` and `Motorcycle` classes using the input provided by the user for make, model, and year.

Call the `GetInfo()` method on each instance to display the vehicle's details.

Instructions:

Write the C# code for the `Vehicle` base class and the derived `Car` and `Motorcycle` classes as described above.

Ensure that the input prompts are clear and informative to guide the user.

Test Case Input:

Toyota
Camry
2020
Honda
CBR600
2019

Expected Output:

Enter car make:
Enter car model:
Enter car year:
Enter motorcycle make:
Enter motorcycle model:
Enter motorcycle year:
Car: 2020 Toyota Camry

In this assignment, you will create a simple student management system using C#. You will define a custom class called Student to represent students and implement basic functionalities to add, view, and update student records.

Task:

1. Define the Student Class:

Create a C# class named Student.

Add properties to the class: Name (string type), Age (integer type), and Grade (string type).

Implement a constructor in the Student class to initialize these properties.

2. Input Handling:

Use the `Console.ReadLine()` method to prompt the user to enter the student's name, age, and grade.

Convert the user input for age to an integer using `Convert.ToInt32()`

method.

3. Create an Instance of the Student Class:

Instantiate the Student class using the input provided by the user for name, age, and grade.

4. Print Output:

Print out the values of the Name, Age, and Grade properties of the created Student instance to the console.

Instructions:

Write the C# code for the Student class and the Main method as described above.

Ensure that the input prompts are clear and informative to guide the user.

Test your program by running it and providing different inputs to verify its functionality.

Test Case Input:

John

18

A

Expected Output:

Enter student's name:

Enter student's age:

Enter student's grade:

Student Name: John

Student Age: 18

Student Grade: A

In this assignment, you will create a basic inventory management system using C#. You will define an array to store product information and implement functionalities to add, view, and update product records.

Task:

1. Define Product Structure:

Create a struct named Product to represent products.

Add fields to the struct: Name (string type), Price (decimal type), and Quantity (integer type).

2. Initialize Product Array:

Create an array of Product structs to store product information.

Define the array size based on the maximum number of products that can be stored in the inventory.

3. Implement Menu Options:

Display a menu with options to add a new product, view all products, and update product details.

Use a switch statement to handle user input and perform corresponding actions.

4. Input Handling:

Prompt the user to enter product details (name, price, quantity) when adding a new product.

Convert the user input for price and quantity to their respective data types.

5. Perform Operations:

Add a new product to the inventory when the user selects the "Add Product" option.

View all products in the inventory when the user selects the "View Products" option.

Update product details (price or quantity) when the user selects the "Update Product" option.

Instructions:

Write the C# code to implement the inventory management system as described above.

Ensure that the menu options are clearly displayed and user-friendly.

Test your program by adding, viewing, and updating product records to verify its functionality.

Document your code with appropriate comments to explain its purpose and functionality.

Test Case Input:

```
1
Apple
1.50
100
2
4
```

Expected Output:

```
1. Add Product
2. View Products
3. Update Product
4. Exit
Enter your choice: Enter product name: Enter product price: Enter product quantity:
Product added successfully!
1. Add Product
2. View Products
3. Update Product
4. Exit
Enter your choice: Name: Apple, Price: 1.5, Quantity: 100
1. Add Product
```

In this assignment, you will create a simple vehicle management system using C#. You will define a base class for vehicles and derived classes for specific types of vehicles, implementing inheritance and method overriding.

Task:

1. Define Vehicle Base Class:

Create a base class named Vehicle to represent general vehicles.

Add properties to the class: Make (string type), Model (string type), and Year (integer type).

Implement a constructor in the Vehicle class to initialize these properties.

Add a virtual method named GetInfo() to display the vehicle's details.

2. Define Derived Classes:

Create a derived class named Car that inherits from Vehicle.

Create a derived class named Motorcycle that inherits from Vehicle.

Override the `GetInfo()` method in both derived classes to include specific details (e.g., type of vehicle).

3. Input Handling:

Use the `Console.ReadLine()` method to prompt the user to enter the make, model, and year of the vehicle.

4. Create Instances and Display Information:

Instantiate the `Car` and `Motorcycle` classes using the input provided by the user for make, model, and year.

Call the `GetInfo()` method on each instance to display the vehicle's details.

Instructions:

Write the C# code for the `Vehicle` base class and the derived `Car` and `Motorcycle` classes as described above.

Ensure that the input prompts are clear and informative to guide the user.

Test Case Input:

Toyota
Camry
2020
Honda
CBR600
2019

Expected Output:

Enter car make:
Enter car model:
Enter car year:
Enter motorcycle make:
Enter motorcycle model:
Enter motorcycle year:
Car: 2020 Toyota Camry
Motorcycle: 2019 Honda CBR600

In this assignment, you will create a simple banking system using C#. You will define classes to represent bank accounts and implement functionalities for deposit, withdrawal, and balance inquiry operations.

Task:

1. Define Account Class:

Create a class named Account to represent bank accounts.

Add properties to the class: AccountNumber (string type), Balance (decimal type), and OwnerName (string type).

Implement access modifiers to control access to account properties.

2. Implement Operations:

Add methods to the Account class for depositing funds, withdrawing funds, and checking account balance.

Use access modifiers to restrict access to sensitive operations like withdrawing funds.

3. Input Handling:

Prompt the user to enter account details (account number, owner name) when creating a new account or performing transactions.

Use appropriate data types for user input and validation.

4. Perform Transactions:

Allow users to deposit funds into their account by specifying the deposit amount.

Allow users to withdraw funds from their account by specifying the withdrawal amount.

Display the updated account balance after each transaction.

Instructions:

Write the C# code to implement the banking system as described above.

Ensure that access modifiers are used appropriately to maintain data privacy and security.

Test your program by creating accounts, depositing funds, and withdrawing funds to verify its functionality.

Document your code with appropriate comments to explain its purpose and functionality.

Test Case Input:

12345

John Doe

Expected Output:

Enter account number:

Enter owner name:

Deposited: \$100.00

Account Balance: \$100.00

Withdrew: \$50.00

Account Balance: \$50.00

GameInheritance

Reena has to teach a lesson on various sports and games that are played around the world , to her grade 4 students..

To make the class interactive, she decided to call each student and ask them the name of a game they knew ,number of players needed to play the game and about games that are played with a time limit.

Help her by writing a C# Program that prompts the user for the said details and display them on the console.

Create the classes , along with the specified members as mentioned below.

1. **class Game** // parent class that describes the games properties

Include **Auto-implemented properties** for the game's name and maximum number of players.

Property Name	Description
public string Name	property to store the name of the game.
public int MaxNumPlayers	Maximum number of players included for the game

Method Name	Description
ToString()	Should overrides the Object class's ToString() method and returns a string that contains the name of the Game, and the number of players as given in the sample output.

2. class GameWithTimeLimit // child class that should inherit class Game

Generate an auto-implemented integer property for Minutes to store the game's time limit in minutes.

Property Name	Description
Public int TimeLimit	store the game's time limit in minutes.

Method Name	Description
ToString()	Should overrides the Object class's ToString() method and returns a string . Should call the base parent class ToString and print the name and number of players . In addition this method should print the time limit for the game. (Refer the sample output.)

3. class Program

Create a class named Program with Main method to instantiate objects for the above mentioned classes and Display the output as given in the sample.

Note:

Do not create any new namespace.

Create classes with public access specifier.

The Main method should be defined in public class Program.

Sample Input:

Enter a game

Cricket

Enter the maximum number of players

11

Enter a game that has time limit

Football

Enter the maximum number of players

11

Enter the time limit in minutes

90

Sample Output:

Maximum number of players for Cricket is 11

Maximum number of players for Football is 11

Time Limit for Football is 90 minutes

OpenableInterface

During the birthday party of Brian, The children were asked to pick a folded sheet of paper with handwritten fortunes inside. If the paper is marked with a letter T, a toy treasure box is gifted, and if it contains a letter P, a toy parachute is gifted to the children.

Simulate the scenario using C# classes and interface.

Create an interface named IOpenable. It should contain a single method named OpenSesame with the following signature.

Interface IOpenable

Member type	Identifier Name	Description
Method	String OpenSesame()	The method contains zero parameter list and returns a string

Create classes named TreasureBox, and Parachute that implements IOpenable.

class TreasureBox // should implement IOpenable

Member type	Identifier Name	Description
Method	String OpenSesame()	method should return a string "Congratulations , Here is your lucky win". (return the string exactly as specified .)

class Parachute // should implement IOpenable

Member type	Identifier Name	Description
Method	String OpenSesame()	method should return a string "Have a thrilling experience flying in air" (return the string exactly as specified .)

class Program // class for the Main method

Member type	Identifier Name	Description
Method	Main	Create instances for Parachute and TreasureBox and call the OpenSesame method to display the fortunes.

Write a program that declares an object for each of the implementing classes and calls its OpenSesame() method . Display the String to the console.

Note:

Don't create any new namespace.

Create classes with public access specifier.

The Main method should be defined in public class Program.

Declare the interface as public

Sample Input

Enter the letter found in the paper

T

Sample Output:

Congratulations, Here is your lucky win

Sample Input

Enter the letter found in the paper

P

Sample Output:

Have a thrilling experience flying in air